GUIA VISUAL 040

SYSTEMD: UNITS E TARGETS

Em vez de um script que faz, um arquivo que descreve — e o PID 1 supervisiona.

040
GUIAS LINUX

ANATOMIA DE UM ARQUIVO `.SERVICE`

[Unit]

```
Description=Coletor
After=network.target
Wants=network.target
```

Quem sou e de quem dependo. Vale para todo tipo de unit.

[Service]

```
Type=simple
ExecStart=/usr/bin/coletor
User=coletor
Restart=on-failure
RestartSec=5s
```

*Como executar e supervisionar. Só existe em **.service**.*

[Install]

```
WantedBy=multi-user.target
```

*Lida **só** no **enable**: diz onde criar o link que faz o serviço subir no boot. Sem ela: No installation config specified.*

TUDO É UNIT: `.service` processo · `.target` marco · `.socket` ativa sob conexão · `.timer` agenda · `.mount` montagem · `.path` vigia caminho · `.device` udev · `.slice` cgroup

REQUISITO × ORDEM — DOIS EIXOS INDEPENDENTES

"B PRECISA DE A?" — REQUISITO

- ✓ `Wants=` puxa A; se A falhar, B sobe assim mesmo. **O padrão sensato.**
- ✓ `Requires=` puxa A; se A falhar ao iniciar, B é desativado.
- ✓ `BindsTo=` se A parar *a qualquer momento*, B para junto.
- ✓ `Conflicts=` iniciar B para A.

"B COMEÇA DEPOIS DE A?" — ORDEM

- ✓ `After=` / `Before=` — e nada mais.
- ✓ Sozinho, `Requires=` **não ordena**: os dois sobem em paralelo.
- ✓ Erro clássico: funciona na máquina lenta, quebra na rápida.
- ✓ Quase sempre se quer o par: `Requires=db.service` + `After=db.service`.

TARGETS SÃO MARCOS · E QUEM VENCE ENTRE OS DIRETÓRIOS

TARGET	MARCO QUE REPRESENTA
`sysinit`	FS, dispositivos, `/tmp`
`basic`	Mínimo pronto; sockets e timers ativos
`network`	Rede configurada — **não** é "tem internet"
`network-online`	Conectividade de fato (custa boot)
`multi-user`	Sistema completo em modo texto
`graphical`	Inclui o multi-user + login gráfico
`rescue` / `emergency`	Shell mínimo / só a raiz somente leitura

`/run/systemd/system/` ↑ vence
tempo de execução — some no reboot

`/etc/systemd/system/`
do administrador — o seu lugar

`/lib/systemd/system/`
dos pacotes — o APT sobrescreve

`…service.d/override.conf`
o *drop-in*: muda só a chave desejada e continua recebendo as correções do pacote. Crie com `systemctl edit` — evite o `--full`.

NA PRÁTICA

kali@kali: ~

```
$ systemctl cat ssh.service      # o arquivo + todos os drop-ins
$ systemctl show ssh -p Type -p Restart   # config efetiva, com os padroes
$ systemd-delta                  # o que neste sistema foge do padrao
$ sudo systemctl edit ssh.service         # cria o drop-in no lugar certo
$ sudo systemctl daemon-reload   # OBRIGATORIO apos editar unit na mao
$ sudo systemctl start tunel@servidor1    # template: %i = servidor1
```

RESUMO RÁPIDO

- ✓ `Type=` diz **como o systemd sabe que subiu**: `simple` (primeiro plano), `forking` + `PIDFile=` (daemon clássico), `oneshot` (roda e acaba).
- ✓ Sem `daemon-reload`, a edição "não faz efeito" — o systemd opera com a versão em memória.
- ✓ *start repeated too quickly* é sintoma, não causa: o serviço falha na partida por outro motivo.
- ✓ Units de usuário em `~/.config/systemd/user/` dispensam `sudo` — substituem o `@reboot` do crontab.

NO KALI

Serviços de rede vêm instalados e **desabilitados** de propósito — máquina de teste vive em rede hostil. Habilitar é ato consciente: `systemctl enable --now ssh`. O par `Requires= + After=postgresql.service` é o caso do Metasploit. E `Type=forking` sem `PIDFile=` é o erro nº 1 ao empacotar ferramenta: se ela tiver `--no-daemon`, use-a com `Type=simple`.